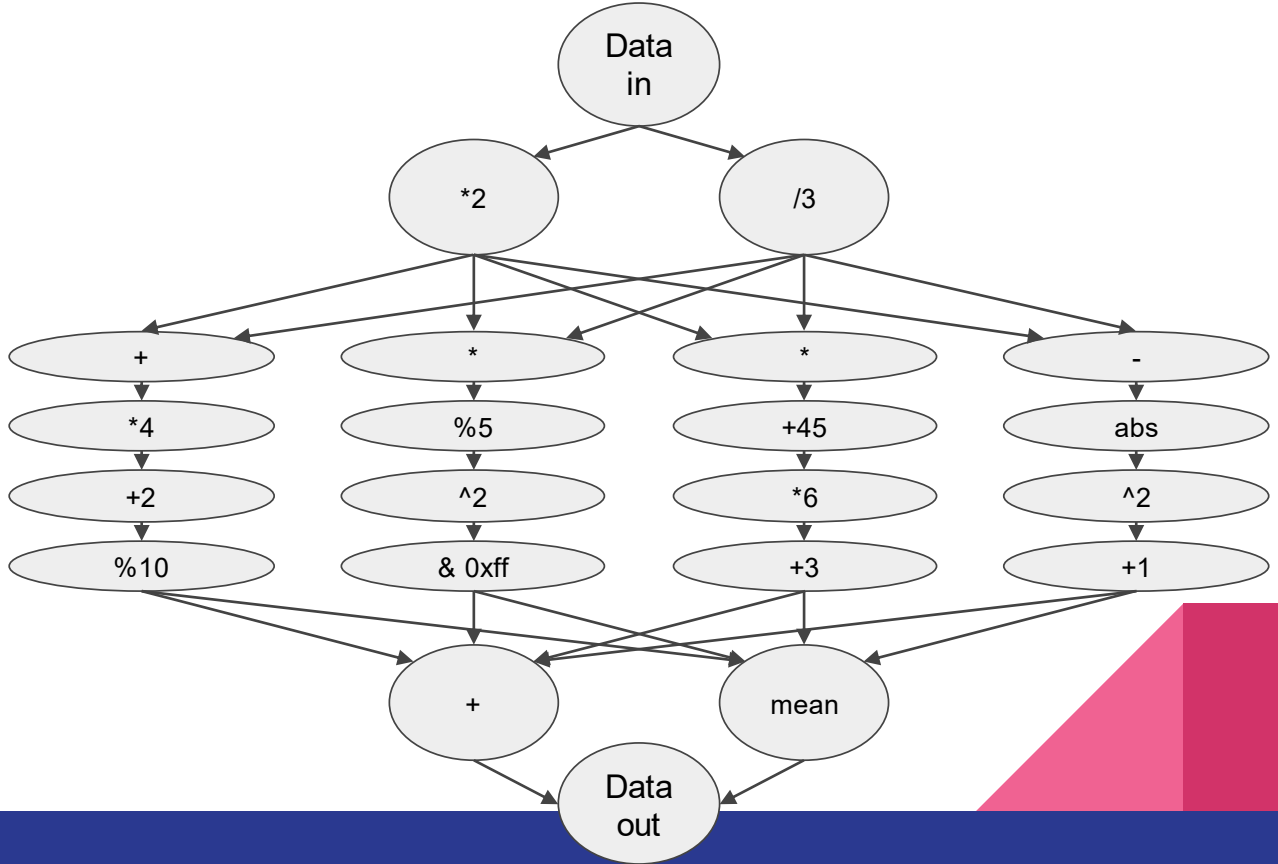
Cooperative Routines

CSE 493L

Agenda

- Coroutines
  - Definition
  - API
  - Many examples
- Channels

How can we build something that runs this?



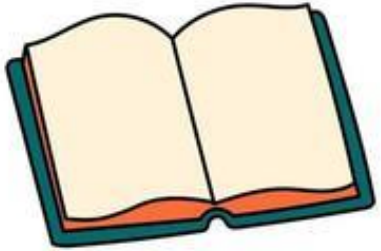
Cooperative Routines

- a.k.a. Coroutines
- An abstraction for expressing concurrent operations
- “Functions that you can pause”
- Today, only one worker or core that has to do everything

But what is a coroutine?

But what is a coroutine?

But what is a coroutine?



But what is a coroutine?

But what is a coroutine?

Ready Coroutine =

Active coroutine =

But what is a coroutine?

Ready Coroutine =

Active coroutine =

Cannot switch away from Active unless Active cooperatively “yields”!

But what is a coroutine?

Ready Coroutine = *2

Active coroutine = +3

Cannot switch away from Active unless Active cooperatively “yields”!

Coroutine API

- yield
  - called from within a coroutine to suspend execution
- spawn
  - create a new coroutine
- join
  - wait for a coroutine to finish

Example 1

```
fn main() {
    PC
        h = spawn(my_routine)

        yield()

        join(h)

}
```

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

Example 1

```
fn main() {

        h = spawn(my_routine)

        yield()

        join(h)

}
```

PC

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

Example 1

```
fn main() {

        h = spawn(my_routine)

        yield()

        join(h)

}
```

```
fn my_routine() {
    PC
        for i in 0..5 {

                print("!")

        }

}
```

Example 1

```
fn main() {
        h = spawn(my_routine)
        yield()
        join(h)
}
```

```
fn my_routine() {
        for i in 0..5 {
                print("!")
        }
}
```

PC

Example 1

```
fn main() {

        h = spawn(my_routine)

        yield()

        join(h)

}
```

PC

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

Example 1

```
fn main() {

        h = spawn(my_routine)

        yield()

        join(h)
```

PC

```
}
```

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

Example 2

```
fn main() {
        h = spawn(my_routine)
        for i in 0..5 {
                print("?")
                yield()
    }
        join(h)
}
```

```
fn my_routine() {
        for i in 0..5 {
                print("!")
                yield()
        }
}
```

Example 2

```
fn main() {
        h = spawn(my_routine)
        for i in 0..5 {
                print("?")
                yield()
    }
        join(h)
}
```

```
fn my_routine() {
        for i in 0..5 {
                print("!")
                yield()
        }
}
```

Options:

?!?!?!?!?!

??!!??!!?!

!?!?!?!?!?

Many flavors of coroutines

- Does yield **have** to switch to a new routine?
- Does spawning a routine **have** to switch to it, or not? Could it?
- Do yields specify a destination routine?
- Can regular functions yield?
- Does yield return data?

Some truths hold constant

- A coroutine will run, uninterrupted, until it gets *blocked* or *voluntarily yields*
  - Example of blocking: join(h)
  - Hence the name “cooperative”: we assume our code voluntarily yields often
  - Otherwise they would be called threads!

What if…

We had an evil coroutine that never called yield?

In our case, for homework 1…

- spawn() can switch to a new coroutine, but it doesn’t have to
- yield() must switch to a different coroutine if possible
  - it returns whether or not it successfully switched!
  - some OSs call this switch()
- join() won’t exist, but you can implement it using Channels

What does join do?

```
fn main() {
    PC
        h = spawn(my_routine)

        join(h)

}
```

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

What does join do?

```
fn main() {

        h = spawn(my_routine)

        join(h)

}
```

PC

```
fn my_routine() {

        for i in 0..5 {

                print("!")

        }

}
```

What does join do?

```
fn main() {

        h = spawn(my_routine)

        join(h)

}
```

```
fn my_routine() {
  PC
        for i in 0..5 {

                print("!")

        }

}
```

What does join do?

```
fn main() {
        h = spawn(my_routine)
        join(h)
}
```

```
fn my_routine() {
        for i in 0..5 {
                print("!")
        }
}
```